

秒杀系统压测总结报告

日期: 2026-06-22
环境: 8.163.136.37 Docker Compose (5 容器)
压测周期: 2026-06-21 ~ 2026-06-22 (3 阶段, 4 轮压测)

1. 压测概览

Phase 1 (初测+重测) ~5min x 2 单接口逐级加压 发现限流瓶颈 限流调整后重测	Phase 2 (混合场景) ~8min 混合流量 1min/级 验证混合场景容量 系统总QPS ~770	Phase 3 (长时驻留) ~25min 混合流量 2min/级+驻留+降载 验证长时间稳定性+MQ ~1M请求 0超卖
--	---	---

阶段	日期	测试模式	时长	总请求	发现问题
Phase 1 初测	06-21	单接口	~5min	~30K	/order 限流 100/s 严重偏低
Phase 1 重测	06-21	单接口 (高限流)	~5min	~30K	/product/active 无缓存导致 P99 1308ms
Phase 2	06-21	混合 1min/级	~8min	~280K	系统总 QPS 天花板 ~770
Phase 3	06-22	混合 2min/级+驻留	~25min	~1,055K	MQ 超时取消机制完全失效

2. 发现的问题汇总

问题 1: /order 限流阈值严重偏低 🚫

发现阶段	Phase 1 初测
现象	c≥50 时 /order 成功率 0%, 5,115 次限流拦截
根因	@RateLimit(rate=100) — 100 QPS 对秒杀业务完全不适用
影响	秒杀核心接口完全不可用, 真实系统容量被隐藏

问题 2: /product/active 无缓存 🚫

发现阶段	Phase 1 重测
现象	c=500 时 P99 飙至 1308ms ，QPS 天花板 540
根因	<code>listActiveProducts()</code> 每次直接查 DB，无缓存层
影响	秒杀列表页（最高频浏览场景）穿透 DB，浪费连接池资源

问题 3：Redis 库存无自动预热

发现阶段	Phase 1 前环境检查
现象	Redis 中无任何 <code>product:stock:*</code> key，Lua 原子扣减形同虚设
根因	SQL 脚本直接 INSERT 的商品不会触发 <code>setProductStock()</code>
影响	高并发下库存扣减直接穿透 DB，行锁竞争加剧

问题 4：热路径使用旧 Lua 脚本

发现阶段	Phase 1 代码审查
现象	<code>decreaseStockWithCache()</code> 使用简单 <code>decrementStock()</code> (GET→检查→DECR)，存在极小并发窗口
根因	系统已准备了更健壮的 <code>checkAndDecrementStock()</code> (DECR+负数回滚)，但热路径从未调用
影响	极端并发下理论存在超卖风险（虽然 Phase 1-3 均未触发）

问题 5：订单初始状态错误

发现阶段	Phase 1 代码审查
现象	<code>order.setStatus(1)</code> 直接将订单标为"已支付"，跳过"待支付"状态
根因	<code>OrderService.java:82</code> 硬编码 <code>status=1</code>
影响	(1) MQ 超时取消逻辑基于 <code>status==0</code> 判断，完全失效；(2) 订单生命周期不完整

问题 6：MQ 超时取消机制完全失效

发现阶段	Phase 3 长时压测
现象	<code>order.dead.letter.queue</code> 全程 0 条消息 ，1498/1500 订单 <code>payExpireTime</code> 过期后仍为 <code>status=0</code>
根因	<code>OrderConsumer.handleOrderCreate()</code> 立即 ACK，消息在队列停留 <1ms， <code>x-message-ttl=600000</code> 永不触发
影响	超时订单永不取消，库存永不释放，死信队列 <code>handleOrderTimeout()</code> 代码正确但不执行

3. 优化措施及前后对比

优化 1：调高 `/order` 接口限流阈值

部署文件: `OrderController.java`, `ProductController.java`

```
- @RateLimit(rate = 100, rateInterval = 1000, ...)
+ @RateLimit(rate = 1000, rateInterval = 1000, ...)
```

指标	优化前 (Phase 1 初测)	优化后 (Phase 1 重测)	改善
c=50 <code>/order</code> 成功 QPS	0	109	∞ (恢复可用)
c=10 <code>/order</code> QPS	99	135	+36%
限流拦截次数	5,115	0	-100%
瓶颈转移	限流层	<code>uk_user_goods</code> 业务层	☑

优化 2： `/product/active` 加 Redis 缓存 (TTL 10s)

部署文件: `ProductService.java`

```
// listActiveProducts() 增加 Redis 缓存层
cacheKey = "product:active:list:" + pageNum + ":" + pageSize;
Object cached = cacheService.get(cacheKey);
if (cached != null) return (IPage<ProductVo>) cached;
// ... query DB ...
cacheService.set(cacheKey, result, 10);
```

指标	优化前 (Phase 1 重测)	优化后 (Phase 2)	改善
c=200 active P99	462ms	935ms (混合竞争)	—
缓存命中响应	123ms (DB)	23ms (Redis)	5.3x
c=500 active P99	1308ms	1652ms (混合竞争)	—

注：Phase 1 为单接口压测，Phase 2 为混合场景，P99 升高是混合竞争导致，非缓存本身问题。
缓存命中响应从 123ms → 23ms 是直接证据。

优化 3：Redis 库存自动预热

部署文件: ProductService.java

```
@PostConstruct
public void warmupStock() {
    List<Product> products = productMapper.selectList(null);
    for (Product p : products) {
        if (p.getStatus() != 3) {
            cacheService.setProductStock(p.getId(), p.getStockCount());
        }
    }
}
```

指标	优化前	优化后
Redis <code>product:stock:*</code> key 数量	0 (需手动预热)	20 (启动自动完成)
首次下单路径	直接 DB UPDATE (行锁竞争)	Redis Lua 原子 DECR → DB
启动日志	无	<code>Redis</code> 库存预热完成，共预热 20 个商品

优化 4：热路径改用 `checkAndDecrementStock`

部署文件: OrderService.java, ProductService.java

```
- cacheService.decrementStock(productId)
+ int result = cacheService.checkAndDecrementStock(productId)
+ // 1=成功, 0=库存不足(自动回滚), -1=key不存在, -2=冲突
```

指标	优化前 (decrementStock)	优化后 (checkAndDecrementStock)
Lua 并发安全	GET→检查→DECR (非原子)	DECR→检查<0→INCR回滚 (原子)
库存回滚	无	自动
超卖风险	理论存在极小窗口	消除
Phase 1-3 超卖	0	0 (此前已安全，但防御增强)

优化 5：修复订单初始状态 (1→0 待支付)

部署文件: OrderService.java

```
- order.setStatus(1);    // 直接已支付
+ order.setStatus(0);    // 待支付，等待支付或15分钟超时自动取消
```

指标	优化前	优化后
新订单状态	1（已支付）	0（待支付）
payExpireTime	有效但被忽略	正常生效
超时取消	永不触发（status≠0 跳过）	可通过定时扫描触发
订单生命周期	不完整	完整：待支付→已支付/已取消

优化 6：MQ 超时订单定时扫描取消

部署文件: OrderService.java, OrderMapper.java

```
@Scheduled(fixedRate = 30000)
public void cancelExpiredOrders() {
    List<Order> expired = orderMapper.selectExpiredOrders(now, 100);
    for (Order o : expired) {
        o.setStatus(2);
        orderMapper.updateById(o);
        productMapper.incrementStock(o.getGoodsId()); // 恢复 DB
        cacheService.incrementStock(o.getGoodsId()); // 恢复 Redis
        cacheService.deleteOrder(o.getId());
        cacheService.deleteProduct(o.getGoodsId());
    }
}
```

指标	优化前	优化后
超时订单状态	status=0（永久停留）	status=2（已取消）
库存释放	✗ 永不释放	☑ ≤30s 内恢复
死信队列	0 条消息（永不触发）	N/A（定时扫描替代）
首次扫描取消数	0	100/100 (Phase 3 遗留)
针对性验证	—	10/10 手动过期→30s 后全取消

4. Phase 1 → 2 → 3 阶段数据对比

4.1 核心性能指标

指标	Phase 1 初测	Phase 1 重测	Phase 2	Phase 3	趋势
测试模式	单接口	单接口	混合 1min/级	混合 2min/级 +驻留	逐步接近 生产
总请求	~30K	~30K	~280K	~1,055K	35x
峰值总 QPS	—	—	772 (@c=200)	777 (@c=100)	一致
稳态总 QPS	—	—	~740 (c≥500)	~670 (c≥500)	竞争增加
c=50 active P99	186ms	168ms	191ms	221ms	稳定
c=200 active P99	582ms	462ms	935ms	988ms	混合竞争↑
c=500 active P99	534ms	1308ms	1652ms	2300ms	DB 层瓶颈
c=1000 active P99	—	—	1635ms	2357ms	饱和态

4.2 数据一致性（所有阶段）

检查项	Phase 1	Phase 2	Phase 3	Phase 4 (优化后)
超卖	0	0	0	0
重复订单	0	0	0	0
Redis/DB 偏差	0	0	0	0
一致性检查次数	~10	~20	123	~10
瞬时 MISMATCH	0	0	33 (事务中间态)	0

4.3 MQ 层

指标	Phase 1	Phase 2	Phase 3	Phase 4 (优化后)
队列积压	0	0	0	0
死信队列消息	0	0	0 (缺陷)	N/A (定时扫描)
超时取消	未测试	未测试	失效	☒ ≤30s

4.4 系统容量评估演进

指标	Phase 1 重测	Phase 2	Phase 3
峰值总 QPS	非混合	~770/s	~777/s
稳态总 QPS	非混合	~740/s	~670/s
安全并发 (P99<1s)	≤100	≤200	≤200
饱和并发 (P99~1.6-2.3s)	≥200	≥500	≥500
极限并发 (不崩溃)	500	1000	1000
10min 驻留验证	无	无	☒ OK
降载恢复	无	无	☒ <1min

5. 瓶颈演进

Phase 1 初测
瓶颈：限流层 (100/s)
↓ 优化 1：限流 100→1000/s
Phase 1 重测
瓶颈：DB 查询层 (/product/active P99 1308ms)
↓ 优化 2：加 Redis 缓存
Phase 2
瓶颈：DB 连接池竞争 (混合场景总 QPS ~770)
↓ 优化 3,4：库存预热 + Lua 升级
Phase 3
瓶颈：DB 连接池竞争 (总 QPS ~670 @ c≥500)
新发现：MQ 超时取消失效
↓ 优化 6：@Scheduled 定时扫描
Phase 4 (当前)
已知瓶颈：DB 连接池 (P99 天花板由 DB 竞争决定)
待优化：HikariCP 连接池大小、复合索引、读写分离

瓶颈	严重度	状态	说明
限流器 (100/s)	●	☒ 已解决	优化 1
/product/active 无缓存	●	☒ 已解决	优化 2
Redis 库存未预热	●	☒ 已解决	优化 3
旧 Lua 脚本	●	☒ 已解决	优化 4
订单初始状态	●	☒ 已解决	优化 5
MQ 超时取消失效	●	☒ 已解决	优化 6
DB 连接池竞争	●	⌚ 待优化	P99 天花板
uk_user_goods 组合耗尽	●	业务特征	测试环境受限
限流器非滑动窗口	●	⌚ 待优化	INCR+固定TTL

6. 架构改进建议

P0 (已完成)

- ✓ 限流阈值调整 (100→1000/s)
- ✓ `/product/active` 加 Redis 缓存
- ✓ Redis 库存自动预热
- ✓ 热路径 Lua 脚本升级 (DECR → checkAndDecrement)
- ✓ 订单初始状态修正 (1→0)
- ✓ MQ 超时订单定时扫描取消

P1 (建议近期执行)

- ☐ **DB 连接池优化**: 当前 HikariCP `maximumPoolSize` 为默认值 10, 混合场景 $c \geq 500$ 时 P99 由连接池等待决定。建议增大到 20-30
- ☐ **pay_expire_time 加复合索引**: `CREATE INDEX idx_status_expire ON seckill_order(status, pay_expire_time)` 提升 `selectExpiredOrders` 扫描效率
- ☐ **`/product/active` 缓存启动预填充**: `@PostConstruct` 预热首页缓存, 消除首次请求 miss

P2 (建议中期执行)

- ☐ **Prometheus + Grafana 监控接入**: 实时监控连接池、GC、QPS、P99, 替代脚本监控
- ☐ **慢查询日志分析**: 开启 MySQL slow query log, 定位具体慢 SQL
- ☐ **读写分离**: `/product/detail`、`/product/active` 等读接口走从库

P3 (建议长期执行)

- ☐ **限流器升级**: INCR+固定 TTL → Redisson RRateLimiter 滑动窗口
- ☐ **MQ 流程重构**: 考虑将 `handleOrderCreate` 改为延迟消费模式, 或完全移除冗余的 MQ 创建消息路径
- ☐ **连接池监控指标暴露**: Micrometer + HikariCP metrics → Prometheus

7. 最终系统状态

秒杀系统最终容量评估		
峰值总 QPS:	~777/s	(c=100, 混合)
稳态总 QPS:	~670/s	(c≥500, 混合)
安全并发范围:	≤200	(P99 < 1s)
饱和并发:	≥500	(P99 ~1.6-2.3s, 稳定运行)
极限并发:	1000	(不 OOM, 不死锁, 不崩溃)
持续稳定性:	10min	@ 800 并发, 0 劣化
降载恢复:	<1min	(P99 2.3s → 450ms)
数据一致性:	100%	(~1.5M+ 总请求, 0 超卖, 0 重复)
库存预热:	自动	(20 商品, @PostConstruct)
DB-Redis 同步:	最终一致	(瞬时偏差 < 1s 内自动恢复)
MQ 超时取消:	≤30s	(@Scheduled 定时扫描)

订单状态流程:	完整	(待支付→已支付/已取消/已退款)	
Lua 原子操作:	增强	(checkAndDecrement + 负数回滚)	

8. 压测数据完整性声明

项目	数量
累计压测轮次	6 轮 (Phase 1 初测、Phase 1 重测、Phase 2、Phase 3 ascent、Phase 3 peak+descent、Phase 4 优化后验证)
累计请求数	~1,568,000
累计订单数	~3,700 (含取消, 已软删除)
发现缺陷	6 个
修复缺陷	6 个
待优化项	7 项
压测脚本	4 个 (phase2_test_v2.py, phase23_test.py, phase3_test.py, phase_opt6_test.py)
监控脚本	1 个 (circuit_breaker.sh)

9. 附录：压测脚本清单

文件	用途	位置
<code>phase23_test.py</code>	Phase 2 混合场景阶梯加压 (1min/级)	本地 <code>mvp/</code> + 服务器 <code>/root/</code>
<code>phase3_test.py</code>	Phase 3 长时驻留+降载 (2min/级)	本地 <code>mvp/</code> + 服务器 <code>/root/</code>
<code>phase_opt6_test.py</code>	Phase 4 优化后验证 (1min/级)	服务器 <code>/root/</code>
<code>circuit_breaker.sh</code>	10s 间隔一致性监控	服务器 <code>/root/</code>
<code>tokens.txt</code>	500 JWT Token 池	服务器 <code>/root/</code>

报告生成时间: 2026-06-22
下次压测建议: 增大 HikariCP 连接池后, 重新执行 Phase 3 级别长时压测, 确认 P99 改善幅度

